

THERMAGUARD AI

Software Requirements Specification

Version 1.0 · September 2026

SIH Problem Statement PS-26162 · Disaster Management

AI-Powered Satellite Intelligence for Industrial Fire Detection & Classification

National Technical Research Organisation (NTRO)

Document	Version	Date	Status
THERMAGUARD SRS	1.0	September 2026	FINAL

1. Project Overview

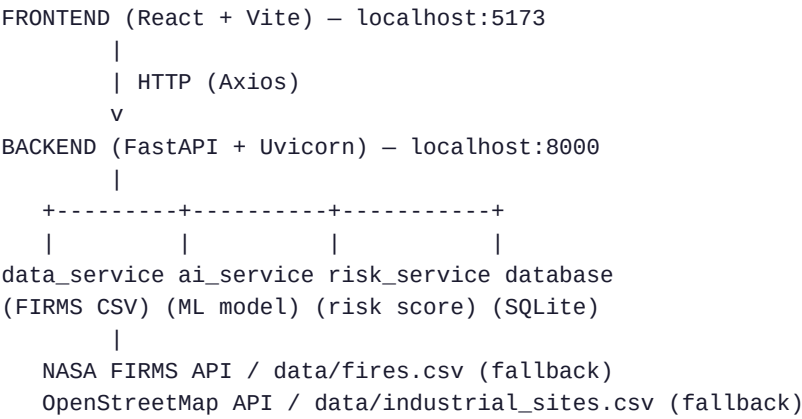
THERMAGUARD AI is an AI-powered satellite thermal intelligence system. It ingests NASA FIRMS satellite data, classifies fire events using a Random Forest ML model, detects persistent thermal sources, calculates risk scores, and displays everything on an interactive GIS map.

Problem Solved

NASA FIRMS satellites detect hotspots but cannot classify them. THERMAGUARD AI classifies each thermal detection into one of five categories:

- 1. Industrial Fire
- 2. Natural / Forest Fire
- 3. Agricultural Fire
- 4. Gas Flare / Persistent Thermal Source
- 5. Other / Unknown

2. System Architecture



3. Functional Requirements

ID	Requirement
FR-01	Ingest thermal events from NASA FIRMS API or CSV fallback
FR-02	Load industrial sites from OpenStreetMap or CSV fallback
FR-03	Classify each event using Random Forest ML (5 classes)
FR-04	Detect persistent sources (≥3 detections within 1.5 km)
FR-05	Calculate risk score (0–100) and risk level per event
FR-06	Display events on interactive Leaflet GIS map
FR-07	Show analytics dashboard with charts and stat cards

ID	Requirement
FR-08	Show alert panel for High and Critical events
FR-09	Support on-demand data refresh
FR-10	Filter events by classification, risk level, persistence, industrial proximity

4. Non-Functional Requirements

Requirement	Target
API response time	< 2 seconds
ML classification latency	< 100 ms per event
Offline mode	Full functionality via CSV fallback
External API key	Optional — not required to run
Database setup	Zero — SQLite auto-created on first run

5. Technology Stack

Backend

- FastAPI 0.115.0 — REST API framework
- Uvicorn 0.30.6 — ASGI server
- SQLAlchemy 2.0.35 — ORM / SQLite
- Pydantic 2.9.2 — data validation
- scikit-learn 1.5.2 — Random Forest ML
- pandas 2.2.3 + numpy 1.26.4 — data processing
- joblib 1.4.2 — model serialisation
- python-dotenv 1.0.1 — config management

Frontend

- React 18.3.1 — UI framework
- Vite 5.4.8 — build tool
- react-leaflet 4.2.1 + leaflet 1.9.4 — interactive GIS map
- Recharts 2.12.7 — analytics charts
- Axios 1.7.7 — HTTP client
- IBM Plex Sans / Mono — typography

ML Model

Parameter	Value
Algorithm	Random Forest
Trees / Max depth	150 trees, max depth 10

Parameter	Value
Features	brightness, FRP, confidence, distance_to_industry, nearby_count, persistence_count, industrial
Output classes	5 fire types
Training samples	2000 synthetic (400 per class)
Model file	models/fire_classifier.pkl

6. API Endpoints

Base URL: <http://localhost:8000> · Interactive docs: <http://localhost:8000/docs>

Method	Endpoint	Description
GET	/health	Server health + demo-mode status
GET	/api/events	All events (filterable)
GET	/api/events/stats	Dashboard summary statistics
GET	/api/events/alerts	High + Critical events only
GET	/api/events/{id}	Single event detail
GET	/api/events/{id}/risk	Risk score + reasons for one event
GET	/api/industrial-sites	All industrial infrastructure
POST	/api/events/analyze	Run full pipeline on ad-hoc coordinate
POST	/api/data/refresh	Re-run ingestion + classification + risk

7. Risk Scoring

Factor	Points
Industrial Fire classification	+30
FRP > 300 MW	+20
FRP 100–300 MW	+10
Distance to industry < 1 km	+25
Distance 1–5 km	+15
Persistent source	+15
Confidence > 80%	+10

Level	Score Range	Colour
Critical	≥ 80	Red

Level	Score Range	Colour
High	≥ 60	Orange
Medium	≥ 35	Yellow
Low	< 35	Green

8. Adding Training Data

Open **data/fires.csv** and add rows in the following format:

```
id,latitude,longitude,brightness,frp,confidence,
acquisition_time,source,true_class_demo_only
TG-9001,22.35,69.82,321.1,47.6,90.6,
2026-09-10T10:00:00Z,MODIS,Industrial Fire
```

Labels must be exactly one of:

- Industrial Fire
- Natural / Forest Fire
- Agricultural Fire
- Gas Flare / Persistent Thermal Source
- Other / Unknown

After adding data: run **python ml/train.py**, then restart the backend.

9. Setup Guide — New Laptop

Step 1 — Install Prerequisites

- Python 3.11+ → <https://python.org/downloads>
- Node.js 18+ → <https://nodejs.org>
- Verify: `python --version` and `node --version`

Step 2 — Copy / Clone Project

- Copy the entire Thermaguard-AI folder, OR
- git clone

Step 3 — Install Backend Dependencies

- `python -m venv .venv`
- `source .venv/bin/activate` # Windows: `.venv\Scripts\activate`
- `pip install -r requirements.txt`
- `cp .env.example .env`

Step 4 — Check / Train ML Model

- Is `models/` # should show `fire_classifier.pkl`
- If missing: `python ml/train.py`

Step 5 — Start Backend

- cd backend
- uvicorn main:app --host 0.0.0.0 --port 8000 --reload
- Wait for: INFO: Application startup complete.
- Backend: <http://localhost:8000> | Docs: <http://localhost:8000/docs>

Step 6 — Install Frontend Dependencies

- cd frontend
- npm install

Step 7 — Start Frontend

- npm run dev
- Wait for: Local: <http://localhost:5173/>
- Open browser: <http://localhost:5173>

Every Time — Run Both Servers

```
Terminal 1 (Backend):
cd Thermaguard-AI/backend
uvicorn main:app --host 0.0.0.0 --port 8000

Terminal 2 (Frontend):
cd Thermaguard-AI/frontend
npm run dev

Open: http://localhost:5173
```

10. Troubleshooting

Problem	Fix
pip not found	Use pip3
python not found	Use python3
Port 8000 busy	Mac: kill \$(lsof -t -i:8000)
ModuleNotFoundError	pip install -r requirements.txt
Map not loading	Check backend is running on port 8000
Model not found	python ml/train.py
npm not found	Install Node.js from nodejs.org